

10.3 RAFT and Memory-Augmented Models

Define RAG in one page, then train the reader with gold plus distractors. Week 14 is the full stack.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Facts can live in **weights** (parametric) or in an **external store** you look up (non-parametric). **RAFT** here is **retrieval-augmented fine-tuning**: you train the model *with* retrieved documents in context so it learns to use them, not only to recite the pretrain snapshot.

Students have not had Week 14 yet. Before RAFT, here is **RAG in one page**. Week 14.1 is the full first-time lecture (architecture, cosine, Lewis's sum, pros and cons). Do not skip that later.

RAG (retrieval-augmented generation) means: look up relevant passages from a collection you control, put them in the prompt, then generate the answer from those passages. The language model is usually **frozen**. You update a fact by editing a document and re-indexing, not by retraining. The index must store **vector + raw text + id** so you can cite. Retrieval can miss; then generation cannot recover. That is why this hour exists: a frozen reader often **ignores** a good snippet. RAFT is how you train the reader.

First time RAFT appears. (RAG itself is defined in one page above, and fully taught in Week 14.1.) **RAFT** is retrieval-augmented **fine-tuning**.

What. Train the reader **with** retrieved snippets in context (gold + **distractors**) so it learns to cite or ignore, not only recite pretrain weights. **Why.** Vanilla RAG at test time with a frozen SFT model often **ignores** the snippet. Index edits then do nothing. **Architecture.** Same retrieve-then-generate stack as RAG. The difference is the **training rows** and that θ **changes**. Loss is SFT on the answer given the bundle. **How.** Build rows: question + top- k texts (some irrelevant) + target answer that uses gold and skips junk. Lab 10's overlap retriever is the toy ranker. **Formula.**

$\mathcal{L} = - \sum_{t \in \text{answer}} \log p_{\theta}(y_t \mid y_{<t}, q, \hat{z}_{1:k})$ with $\hat{z}$ containing distractors on purpose. **Tradeoffs.** + Reader learns to use the open book. – Still SFT (can copy); needs a retriever at train time; not a new architecture; gold-only rows never practice skip.

1. RAFT: train the reader, not only the index

A vanilla RAG pipeline (Week 14) retrieves at test time and hopes the frozen LLM attends. Domain RAFT (Zhang et al. and related recipes) builds training rows that look like the test stack: question, a set of snippets (some relevant, some **distractors**), and an answer that must cite or ignore on purpose. The loss is SFT on that answer. The model learns "read this, skip that."

RAFT: retrieval is in the training loop, not only at demo time



If every training snippet is gold, the model never practices rejecting junk. Include negatives. If you never train with retrieval, editing the index will not fix a model that ignores the index. RAFT is still SFT: the labels are answers *given the bundle*, not a new architecture.

Stanford CS224N 2025 L13: **parametric** recall versus **open-book**. Week 14 is the RAG pipeline (index, retrieve, generate). This hour is the training distribution that makes the reader use the open book instead of the pretrain snapshot.

2. Parametric versus non-parametric memory

Parametric: the MLP and attention weights. Fast, always on, hard to update (ROME, SFT, forgetting).

Non-parametric: a datastore, BM25, embeddings, a kNN over hidden states (kNN-LM, RETRO, memorizing transformers). You update a fact by writing a snippet, not by locating a layer.

Two stores. Editing one does not automatically fix the other.



Hybrids are the product default: parametric priors plus retrieved evidence. Lab 10's token-overlap retriever is a non-parametric cartoon. Overwriting a dict key is a parametric cartoon.

Retrieve-then-answer is a **stack**: index → top- k → prompt → SFT loss on the answer. RAFT is the training distribution for that stack (gold + distractors). Vanilla RAG is often the same stack with a frozen reader. Do not mix the words.

3. What to choose

Need daily-changing policy PDFs? Non-parametric plus a RAFT-style reader. Need style and instruction following? Parametric SFT/LoRA. Need to correct one biography? Try an edit **and** a datastore row; measure both. Hallucination often means the parametric prior won over the snippet. RAFT is how you train that fight.

If $k = 2$ and gold is ranked 3rd, retrieval failed before the reader ran. Raise k , fix chunking, or change overlap. If gold is in the prompt and the answer still recites the pretrain snapshot, that is a **reader** bug: RAFT's job.

4. Teaching this note

About **35 minutes** at the board: retrieve-then-answer on one question with 1 gold + 3 distractors, then parametric vs a PDF you can edit. Play Karpathy **27:43–33:32** (tool use) and mention **40:45** (custom GPTs / files as retrieval). Lab 10 section 2 is token-overlap retrieve.

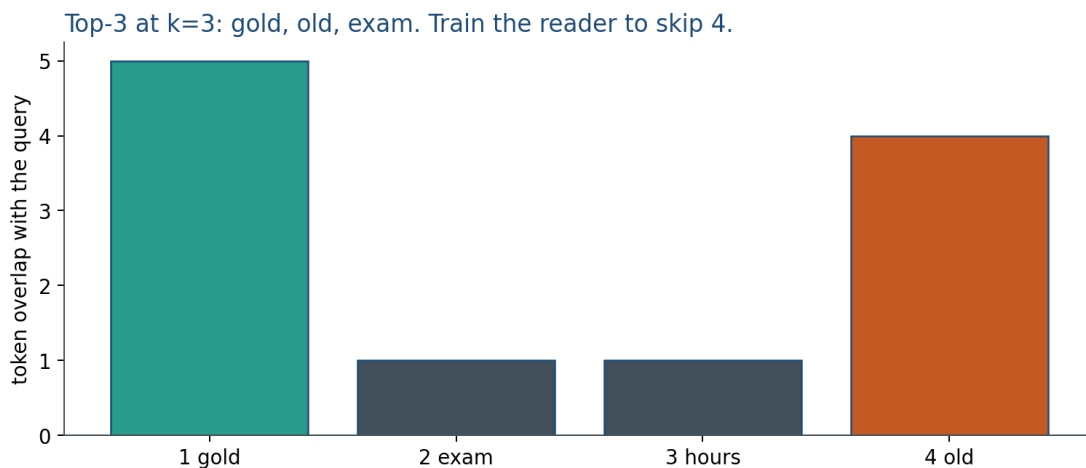
Count overlap tokens live for two snippets so retrieval is a number, not a slogan. Then ask: if gold is in the prompt and the answer is still the old salary, whose bug is that?

5. Worked example

Question: "What is the 2026 lab late policy?" Index has four snippets:

1. Gold: "Labs submitted after Sunday 11:59 pm lose 10% per day."
2. Distractor: "The **exam** is closed book."
3. Distractor: "Office hours are Tuesday."
4. Old: "Labs were due Friday" (wrong year).

Retrieve top-3 by token overlap with the question. Suppose scores (shared tokens) are 5, 1, 1, 4. Top-3: snippets 1, 4, 2. A RAFT **training** row is: question + those three texts + target answer that cites snippet 1 and **ignores** 4. If you only ever trained with snippet 1 alone, the model never learned to skip 4.



At test time the same retrieve-then-answer stack runs. If the reader was frozen SFT with no context, it may still say “Friday” from weights. That failure is not an index bug.

Overlap count is a toy retriever (Lab 10). Cosine on embeddings is the grown-up version (Week 14). RAFT does not care which retriever you used; it cares that **training rows look like test rows**, distractors included.

Update a salary in a PDF and re-index: you changed **non-parametric** memory. ROME on “Alice’s salary” would be parametric. Prefer the PDF if the number moves every month.

6. Where students get stuck

- Training RAG readers on gold-only context, then blaming the index when distractors win.
 - Editing weights to fix a fact that should have been a datastore row.
 - Calling any retrieve-then-generate “RAFT.” RAFT is **training** the reader on that bundle.
-

7. Video

[Karpathy — Intro to Large Language Models](#). Play **27:43–33:32** (tools: browser, calculator, interpreter — retrieval as a tool) and nod to **40:45** (files / RAG-style customization). Pause when the model looks something up instead of “remembering.” RAFT is how you SFT that habit. Week 14 is the full RAG pipeline.

8. Practice

1. Why add distractor snippets to RAFT training rows?
2. You can fix a wrong salary in a PDF. Which memory did you update?
3. Name one failure that retrieval cannot fix if the reader was never trained to look at the context.

4. Four snippets, overlap counts 2, 9, 0, 7. You retrieve top- $k = 2$. Which ids go into the prompt? If gold is id 2, did retrieval succeed?
5. A RAFT row has 1 gold and 3 distractors. The target answer quotes the gold. What should the loss do if the model copies a distractor sentence instead? Why is that different from ordinary SFT on Wikipedia?